CScript

Table of Contents

- Getting Started
    - Hello, World!
- Global Variable Declarations
- Functions Declarations
- Loops
- Conditional Statements
- Switch Statements
- Comments
- Control Transfer Statements
- Statement Termination
- Data Types
- User-defined data types
- Pointer dereferencing
- Arrays
- Operators
- sizeof keywords
- Input Format
- Output

Getting Started:

```
print("Hello, World!");
```

Global Variable Declarations (Normal and Extern):

The language supports global variables, which are variables that are declared outside of any function. Global variables are accessible throughout the program, including inside of any function.

The language also supports external global variables using the `extern` keyword. An `extern` variable represents a global variable that is declared but not defined in the current file.

We use var for normal variable declarations.

```
var count;

var rate = 3.5;

var flag;

var a, b, c;

extern var total;

extern var average;
```

Function declarations:

The language supports function declaration, function definition, and function invocation (function call). Functions may take zero or more parameters and may optionally return a value.

Recursive function calls are permitted, allowing a function to call itself directly or indirectly.

```
/* Function declaration */

func add(var x, var y);
```

```
func factorial(var n) {
    if (n <= 1) {
```

```
        return 1;
    }
    return n * factorial(n - 1);
}
```

Loops:

The language supports iteration constructs to repeatedly execute a block of code based on a condition. The supported loop constructs are:

`for` loop

```
for (var i = 0; i < 5; i = i + 1) {

}
```

`while` loop

```
var i = 0;

while (i < 5) {

    i = i + 1;

}
```

`do-while` loop

```
var i = 0;
do {
    i = i + 1;
} while (i < 5);
```

Conditional Statements:

The language supports conditional execution using `if`, `if-else`, and `if-else-if` constructs. Conditions may be any valid expression, including literals and constants.

Nested conditionals and if-else-if ladders are fully supported.

```
if ( <expression> ) {
    <statement_list>
}
else if ( <expression> ) {
    <statement_list>
} else {
    <statement_list>
}
```

Switch Statements:

The language supports multi-way conditional branching using the `switch` statement.
The `switch` statement selects one of several execution paths based on the value of an expression.

Each `case` label is followed by a colon (`:`).
The language supports fall-through behavior between cases and an optional `default` clause.

Switch with fall-through:

```
var x = 2;
switch (x) {
    case 1:
        print("One");
    case 2:
        print("Two");
    case 3:
        print("Three");
}
```

Switch with optional default clause:

```
var day = 7;
switch (day) {
    case 1:
        print("Monday");
    case 2:
        print("Tuesday");
    case 3:
```

```
        print("Wednesday");
    default:
        print("Invalid day");
}
```

Comments:

The language supports comments to allow explanatory text within programs. Comments are ignored by the language processor and do not affect program execution.

The language supports both single-line and multi-line comments.

```
var x = 10;  // this is a single-line comment
/*
 This is a multi-line comment.
 It can span multiple lines.
*/
var y = 20;
```

Control Transfer Statements:

The language supports control transfer statements that alter the normal flow of execution.
The supported control transfer statements are:

`break`

```
var i = 0;

while (1) {

    if (i == 5) {

        break;

    }

    i = i + 1;

}
```

`continue`

```
for (var i = 0; i < 5; i = i + 1) {

    if (i == 2) {

        continue;

    }

    print(i);

}
```

`return`

```
func add(var a, var b) {

    return a + b;

}
```

Statement termination:

Every executable statement must terminate with a semicolon (`;`), similar to the convention used in the C programming language.

Valid Statements:

```
var x = 10;

x = x + 1;

print(x);

return x;

break;

continue;
```

Data types:

The language supports a set of primitive data types similar to those found in C-like languages.
However, data types are not explicitly declared in the syntax.

Supported Data Types

```
int
float
bool
char
null
```

```
var num = 42;              /* Number */

var isTrue = true;         /* Boolean */

var text = "Hello";        /* String */

var num = null;             /* null */
```

User-defined data types (struct):

The language supports **user-defined structured data types** using the `struct` construct.
A structure groups multiple variables under a single logical unit.

Structure Definition:

```
struct Point {
    var x;
    var y;
};
```

Structure Member access:

```
p.x = 10;
p.y = 20;
```

Pointer Dereferencing:

The language supports pointer dereferencing using the unary * operator. Arbitrary levels of indirection are permitted syntactically.

```
***ptr = 5;
```

Arrays (Derived Type):

Arrays store multiple values accessed using an index expression.

```
var arr[3];
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
```

```
var nums[4] = {1, 2, 3, 4};
```

Strings:

The language supports string literals using the var keyword. Strings may be concatenated using the + operator to form a new string literal.

```
var <identifier> = "<string_literal>" ;
<string_expression> + <string_expression>
var a = "Hello";
var b = "World";
var c = a + " " + b;
```

Operators:

The language supports a variety of operators for arithmetic, relational comparison, logical evaluation, assignment, and data access.

Operator precedence and associativity follow the same rules as in C.

Relational operators:

```
<   >   <=   >=   ==   !=
```

For ex

```
if (a >= b) {
    print(a);
}
```

Unary operators:

```
+   -   !   ~   *   &
```

+, - : unary plus and minus

! : logical negation

~ : bitwise NOT

* : pointer dereference

& : address-of

Binary operators:

```
+   -   *   /   %   &   |   ^
```

Arithmetic: + - * / %

Bitwise: & | ^

Example

```
var a = 10 + 5;

var b = a * 2;

var c = a & b;
```

Logical operators:

Used for boolean operators && ||

Assignment operator:

=

Increment and Decrement operator:

Only postfix form is supported.

```
i++;
j--;
```

Structure field Access operator:

. is used for direct structure member access

-> is used for pointer-based structure access

```
p.x = 10;
```

```
q->y = 20;
```

Pointer operator

```
*    &
*****ptr = 5; // For example
```

Function call operator

```
( )
// For example
add(3, 4);
print("Hello");
```

Array subscript operator

```
[ ]
// For example
arr[i + 1] = arr[i];
```

SizeOf operator:

The language supports the **`sizeof`** operator, which returns the size of its operand.

Unlike C, the `sizeof` operator in this language does not operate on explicit type names, since the language does not expose explicit data types in its syntax.

Instead, `sizeof` operates on expressions only.

```
sizeof
// For example
sizeof ( <expression> )
// Ex1
var x;
var s = sizeof(x);
// Ex2
var arr[10];
var s = sizeof(arr[0])
```

Running your own code:

To run code in CScript, follow these steps:

1) Write the source code in a .cs file

2) The source file must be provided as a **command-line argument** to the executable. If the input program conforms to the language specification, it will be processed successfully

```
$ ./a.out file.cs
```